PCET's Pimpri Chinchwad University Learn | Grow | Achieve	**PIMPRI CHINCHWAD UNIVERSITY** **School of Engineering and Technology**

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Assignment No. 1 a

Title: Write a program to generate Fibonacci series using recursion function.

1) **Objective:**
   - To understand the concept of the Fibonacci sequence.
   - To implement and analyze different approaches for generating the Fibonacci sequence.
2) **Outcomes:**
   - At the end of this lab, students will be able to apply the Fibonacci sequence in programming.
   - Demonstrate the concept of overlapping subproblems and improve efficiency using memorization in Fibonacci computation.
   - Analyze the time complexity of recursive and iterative Fibonacci algorithms to understand the importance of optimization.
3) **Aim:**

   To study the Fibonacci sequence and analyze its role in algorithm efficiency, recursion, and optimization techniques.
4) **Theory:**

   Fibonacci Sequence

   The Fibonacci sequence is a series of numbers in which each number (after the first two) is the sum of the two preceding ones. It usually starts with 0 and 1. The sequence is widely used in mathematics, computer science, and even in nature.

   Mathematical Definition:

   The Fibonacci sequence can be defined using the following recurrence relation:
   - $F(0) = 0$
   - $F(1) = 1$
   - $F(n) = F(n-1) + F(n-2)$ for $n > 1$

   The first few terms of the Fibonacci sequence are:

   0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, ...

- **Basic Operations of Fibonacci Sequence**

1. **Generate Fibonacci Series** - Generate a list of Fibonacci numbers up to a given limit or position.
2. **Fibonacci Number Calculation (n-th number)** - Calculate the n-th Fibonacci number in the sequence.

PCU

3. **Matrix Exponentiation for Fibonacci** - Use matrix exponentiation to find Fibonacci numbers in logarithmic time.
4. **Memoization or Dynamic Programming** - Store previously computed Fibonacci numbers to avoid redundant calculations.

- **Algorithm for Generating Fibonacci Sequence:**
  1. Initialize two variables a = 0 and b = 1.
  2. Print a (the first Fibonacci number).
  3. For a given number of terms, repeat the following steps:
     - Print b (the current Fibonacci number).
     - Set a to the previous b.
     - Set b to the sum of the previous a and b.
  4. Stop when the desired number of terms is printed.
- **Algorithm for Calculating the n-th Fibonacci Number (Using Recursion):**
  1. If n == 0, return 0.
  2. If n == 1, return 1.
  3. Otherwise, return the sum of the Fibonacci number at n-1 and n-2, i.e., F(n) = F(n-1) + F(n-2).
- **Algorithm for Fibonacci Number using Dynamic Programming (Memoization):**
  1. Create an array fib[] to store Fibonacci numbers.
  2. Set fib[0] = 0 and fib[1] = 1.
  3. For each i from 2 to n, calculate fib[i] = fib[i-1] + fib[i-2].
  4. Return fib[n].
- **Algorithm for Fibonacci Number using Matrix Exponentiation:**
  1) Represent the Fibonacci sequence using a matrix multiplication:

$$\begin{bmatrix} F(n) \\ F(n-1) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{(n-1)} \times \begin{bmatrix} F(1) \\ F(0) \end{bmatrix}$$

  2) Compute the matrix exponentiation using the fast exponentiation technique.
  3) The top element of the resulting matrix will be the nth Fibonacci number.
- **Applications of Fibonacci Sequence:**

  The Fibonacci sequence appears in many different fields, including:

  1. **Mathematics and Number Theory:**

PCU

- The Fibonacci sequence is fundamental in number theory and combinatorics, such as counting problems.
- Fibonacci numbers are used in the study of the golden ratio, as the ratio between successive Fibonacci numbers approximates the golden ratio (approximately 1.618).

2. **Computer Science:**
   - Fibonacci numbers are used in algorithms such as dynamic programming and divide-and-conquer algorithms, where sub-problems are solved recursively.
   - Fibonacci heaps are a data structure used in graph algorithms, particularly in Dijkstra's shortest path algorithm.
3. **Nature:**
   - The Fibonacci sequence appears frequently in nature, such as in the arrangement of leaves on a stem, the branching of trees, or the arrangement of seeds in a sunflower.
   - The spiral patterns in seashells and galaxies also follow the Fibonacci sequence.
4. **Finance and Trading:**
   - In technical analysis for stock market predictions, Fibonacci retracement levels are used to predict the potential reversal points of an asset's price.

<u>Source Code : Using Recursion</u>

```
#include <iostream>
using namespace std;

// Recursive function to calculate Fibonacci using if-else
int fibonacci(int x) {
  if (x == 0 || x == 1) {
     return x;
  } else {
     return fibonacci(x - 1) + fibonacci(x - 2);
  }
}
int main() {
```

```
  int terms = 10;
  cout << "Fibonacci Series using recursion and if-else: ";
  for (int i = 0; i < terms; i++) {
     cout << fibonacci(i) << " ";
  }
  return 0;
}
```

Source Code : Without Recursion

```
#include <iostream>
using namespace std;
// Class to generate Fibonacci sequence
class Fibonacci {
public:
   // Function to generate Fibonacci sequence up to n terms
   void generate(int n) {
      if (n <= 0) {
         cout << "Please enter a positive integer." << endl;
         return;
      }
      int first = 0, second = 1;
      cout << "Fibonacci Sequence: ";
      for (int i = 1; i <= n; i++) {
         cout << first << " ";
         // Calculate the next term
         int next = first + second;
         first = second;
         second = next;
      }
      cout << endl;
   }};
```

```
int main() {
    // Initialize a Fibonacci object
    Fibonacci fibonacci;
    int n;
    while (1) {
        cout << "\n**** MENU ****";
        cout << "\n1: Generate Fibonacci Sequence 2: EXIT
        cout << "\nEnter Your Choice: ";
        int ch;
        cin >> ch;
        switch (ch) {
        case 1:
            cout << "Enter the number of terms: ";
            cin >> n;
            fibonacci.generate(n);
            break;
        case 2:
            return 0;
        default:
            cout << "Invalid choice. Please try again." << endl;
        }
    }
    return 0;
}
```

<u>Output of Fibonacci Sequence:</u>

**** MENU ****

1:Generate Fibonacci Sequence

2:EXIT

Enter Your Choice:1

PCU

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

```
Enter the number of terms:5

Fibonacci Sequence: 0, 1, 1, 2, 3

**** MENU ****

1:Generate Fibonacci Sequence

2:EXIT

Enter Your Choice:1

Enter the number of terms:7

Fibonacci Sequence: 0, 1, 1, 2, 3, 5, 8

**** MENU ****

1:Generate Fibonacci Sequence

2:EXIT

Enter Your Choice:1

Enter the number of terms:10

Fibonacci Sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

**** MENU ****

1:Generate Fibonacci Sequence

2:EXIT

Enter Your Choice:2
```

Questions:

1) What is the Fibonacci series, and how is it defined?
2) Write the first 10 terms of the Fibonacci series.
3) Write a simple pseudocode to generate the Fibonacci series up to the nth term without using recursion?

PCU

PCET's Pimpri Chinchwad University
Learn | Grow | Achieve

PIMPRI CHINCHWAD UNIVERSITY
School of Engineering and Technology

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Assignment No. 1 b

Title: Write a Program to sort an array of N elements using Heap sort.

1) Objective:

- To understand the Heap Sort and its implementation.

2) Outcomes:

- At the end of this lab, the students will be able to implement Heap sort.
- Use of arrays to carry out Heap sort.

3) Aim:

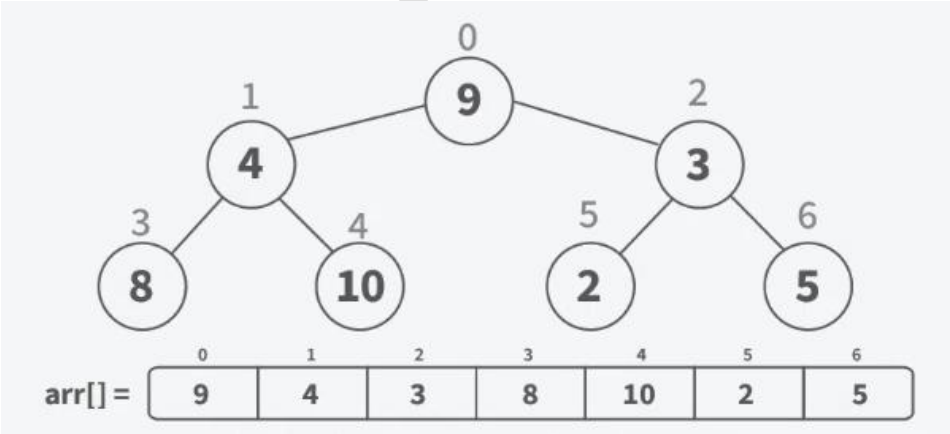
Write a Program to sort an array of N elements using Heap sort.

4) Theory:

Heap Sort is a comparison-based sorting algorithm that utilizes a binary heap data structure to sort elements. It is similar to selection sort, where we first find the maximum element and place it at the end. We repeat the same process for the remaining elements.

How Heap Sort Works

1. **Build a Max Heap:** Convert the unsorted array into a max heap, where the largest element is at the root.
2. **Treat the Array as a Complete Binary Tree**



3. **Extract Maximum Element:** Swap the root (maximum element) with the last element of the heap and reduce the heap size by one.
4. **Heapify:** Reconstruct the heap to maintain the max heap property.
5. **Repeat:** Continue extracting the maximum element and heapifying until all elements are sorted.

Time Complexity

- **Best Case:** O(N log N) — Heap construction and extraction take logarithmic time.
- **Average Case:** O(N log N) — Each insertion and deletion takes O(log N) time.
- **Worst Case:** O(N log N) — Even for reverse-sorted data, heap sort maintains its efficiency.

Space Complexity

O(1) — Heap Sort is an in-place sorting algorithm, requiring only a constant amount of additional space.

Applications of Heap Sort algorithm

- **Priority Queues:** Used in data structures that require constant-time access to the largest/smallest element.
- **Operating Systems:** Utilized in scheduling algorithms like heap-based priority scheduling.
- **Graph Algorithms:** Applied in Dijkstra's shortest path algorithm and Prim's minimum spanning tree.
- **Real-time Applications:** Suitable for time-sensitive systems due to its predictable worst-case performance.

Steps of the Algorithm:

- Convert the array into a max heap using the heapify process.
- Extract the maximum element (root of the heap) and swap it with the last element.
- Reduce the heap size and heapify the new root to maintain the heap property.
- Repeat the process until all elements are extracted, resulting in a sorted array.

Source Code :

```
#include <iostream>

using namespace std;

void heapify(int arr[], int n, int i) {

   int largest = i;

   int left = 2 * i + 1;

   int right = 2 * i + 2;

   if (left < n && arr[left] > arr[largest])

      largest = left;
```

```
    if (right < n && arr[right] > arr[largest])
        largest = right;
    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }}
void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }}
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);
    heapSort(arr, n);
    cout << "Sorted array: ";
    printArray(arr, n);
```

PCU

```
    return 0;

}
```

Output :

```
Sorted array: 5 6 7 11 12 13
```

Algorithm for Heap Sort

1. Start: Initialize an array of nelements.
2. Build Max Heap: Convert the array into a max heap.
   a. Start from the last non-leaf node and heapify each node up to the root.
3. Heap Sort Process: For each element from n−1n-1n−1 to 1:
   a. Swap the root (maximum element) with the last element.
   b. Reduce the heap size by one.
   c. Heapify the root to maintain the max heap property.
4. Repeat: Continue this process until all elements are extracted.
5. End: The array is now sorted in ascending order.

Conclusion:

Thus, we have sorted an array of N elements using Heap sort using an array.

Questions:-

1) What is heap sort, and how does it work?

2) What is the time complexity of heap sort in the best, worst, and average cases?

3) What are the advantages and disadvantages of heap sort?

PCU

PCET's Pimpri Chinchwad University Learn | Grow | Achieve	**PIMPRI CHINCHWAD UNIVERSITY** **School of Engineering and Technology**

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Assignment No. 2 a

a. **Title: Write a program that uses both recursive and non-recursive functions to perform the following searching operations for a key value in a given list of integers:**

- **Binary Search.**

1) Objective:

- To understand Binary Search and its implementation.

2) Outcomes:

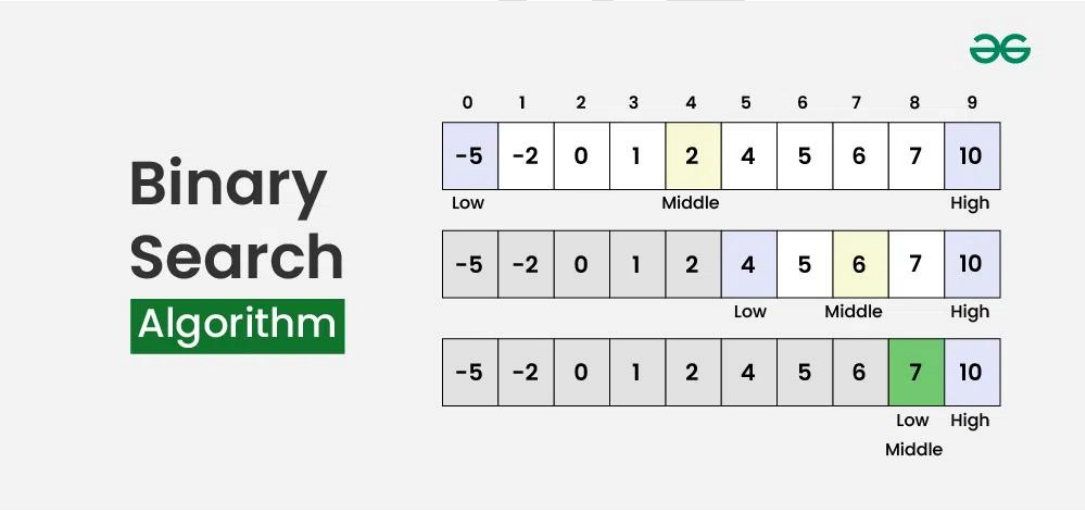
- At the end of this lab, the students will be able to implement Searching Operations.
- Use of arrays to carry out Binary Search.

3) Aim:

Write a program to search an element in the array of N elements using Binary search.

4) Theory:

Binary Search Algorithm is a searching algorithm used in a sorted array by repeatedly dividing the search interval in half. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to O(log N).



What is Binary Search Algorithm?

Binary search is a search algorithm used to find the position of a target value within a sorted array. It works by repeatedly dividing the search interval in half until the target value is found or the interval is empty. The search interval is halved by comparing the target element with the middle value of the search space.

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Conditions to apply Binary Search Algorithm in a Data Structure

To apply Binary Search algorithm:

- The data structure must be sorted.
- Access to any element of the data structure should take constant time.

Binary Search Algorithm

Below is the step-by-step algorithm for Binary Search:

- Divide the search space into two halves by finding the middle index "mid".
- Compare the middle element of the search space with the key.
- If the key is found at middle element, the process is terminated.
- If the key is not found at middle element, choose which half will be used as the next search space.
  - If the key is smaller than the middle element, then the left side is used for next search.
  - If the key is larger than the middle element, then the right side is used for next search.
- This process is continued until the key is found or the total search space is exhausted.

Source Code:

```
def binarySearch(arr, low, high, x):
    while low <= high:
        mid = low + (high - low) // 2
        if arr[mid] == x:
            return mid
        elif arr[mid] < x:
            low = mid + 1
        else:
            high = mid - 1
    return -1
if __name__ == '__main__':
    arr = [2, 3, 4, 10, 40]
    x = 10
    result = binarySearch(arr, 0, len(arr)-1, x)
    if result != -1:
        print("Element is present at index", result)
```

PCU

```
    else:
        print("Element is not present in array")
```

Output:

main.py | Share | Run | Output

```
def binarySearch(arr, low, high, x):
    while low <= high:
        mid = low + (high - low) // 2
        if arr[mid] == x:
            return mid
        elif arr[mid] < x:
            low = mid + 1
        else:
            high = mid - 1
    return -1
if __name__ == '__main__':
    arr = [2, 3, 4, 10, 40]
    x = 10
    result = binarySearch(arr, 0, len(arr)-1, x)
    if result != -1:
        print("Element is present at index", result)
    else:
        print("Element is not present in array")
```

```
Element is present at index 3

=== Code Execution Successful ===
```

Conclusion:

Thus, we have implemented Linear and Binary Search using array.

Questions:-

1) What is binary search, and how does it work?
2) What are the prerequisites for applying a binary search algorithm on a dataset?
3) What is the time complexity of binary search, and how does it compare to linear search?

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Assignment No. 2 b

b. Title: Write a Program to sort an array of N elements using Quick sort.

1) Objective:

- To understand the sorting concepts.
- To understand the basic operations of Quick sort.

2) Outcomes:

- At the end of this lab, the students will be able to use array for Quick sort.
- Quick sort helps to organize the large datasets and make them more accessible.

3) Aim:

Write a Program to sort an array of N elements using Quick sort. Also write how to determine time complexity and space complexity of your program.

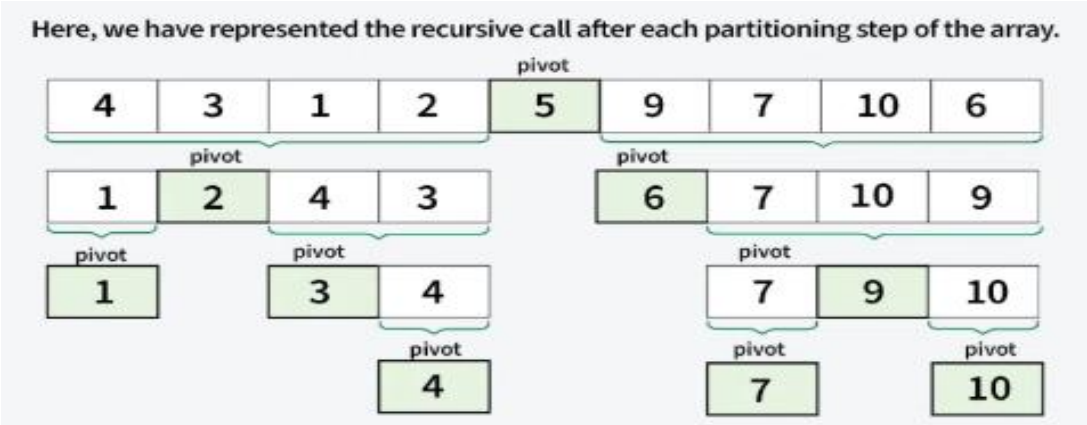
4) Theory:

Quick Sorting refers to the process of rearranging elements present in a data structure in ascending or descending order. The algorithms that achieve this task are known as sorting algorithms. The need is to find an algorithm that produces an ordered structure in minimal time and space.

Quick Sort is an efficient, comparison-based sorting algorithm that uses the divide-and-conquer approach to sort an array or list of elements. This approach consists of the following main steps:

- **Divide:** The algorithm selects a pivot element and partitions the array into two subarrays, placing elements smaller than the pivot on the left and larger ones on the right.
- **Conquer:** It recursively sorts the resulting subarrays
- **Combine:** Since subarrays are sorted in place, no explicit combination step is needed.

Here's a basic overview of how the Quick Sort algorithm works.



Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Algorithm:

1. Select an element as a pivot (typically the last element of the array).
2. Partition the array such that elements smaller than the pivot are moved to the left, and elements greater than the pivot are moved to the right.
3. Recursively apply the above steps to the left and right subarrays until the base case (single-element array) is reached.
4. The array is now sorted.

Time Complexity:

- **Best-case:** O(N log N)
- **Average-case:** O(N log N)
- **Worst-case:** $O(N^2)$ (when the smallest or largest element is always chosen as the pivot)

Space Complexity:

- O(log N) (for recursive stack calls)

Applications of Quick Sort:

- Used in commercial computing.
- Used in numerical computations.
- Used in information searching.

Source code:

```
#include<iostream>
using namespace std;
// Function to swap two elements
void swap(int& a, int& b) {
   int temp = a;
   a = b;
   b = temp;
}
// Partition function
int partition(int arr[], int low, int high) {
   int pivot = arr[high]; // Choosing the last element as pivot
   int i = (low - 1);
```

PCU

```
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return (i + 1);
}
// Quick Sort function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1); // Sort elements before partition
        quickSort(arr, pi + 1, high); // Sort elements after partition
    }
}
// Function to display the array
void displayArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}
// Main function
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
```

```
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    quickSort(arr, 0, n - 1);
    cout << "Sorted array: ";
    displayArray(arr, n);
    return 0;
}
```

Output:

Enter the number of elements: 7

Enter 7 elements: 21 13 20 14 17 11 4

Sorted array: 4 11 13 14 17 20 21

Questions:-

1) What is the quick sort algorithm, and how does it work?
2) Explain the role of the pivot element in the quick sort algorithm.
3) Why is quick sort considered a divide-and-conquer algorithm?

PCU

PCET's Pimpri Chinchwad University Learn | Grow | Achieve	**PIMPRI CHINCHWAD UNIVERSITY** **School of Engineering and Technology**

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Assignment No. 3

Title: Write a program to solve job sequencing using deadline using Divide and Conquer approach.

Objective:

- To understand how to schedule jobs to meet deadlines while maximizing profits.
- List. To implement job sequencing using efficient data structures like linked lists.

1) Outcomes:

- Students will be able to implement job sequencing algorithms to optimize task scheduling based on deadlines.
- Students will learn to handle and manipulate data using linked lists to efficiently manage sequential job data.

2) Aim:

To implement a job sequencing algorithm using deadlines in a program (preferred language by student), incorporating operations like insertion and deletion on singly and doubly linked lists to manage job data effectively.

3) Theory:

Job sequencing using deadlines is a vital concept in scheduling and optimization. The goal is to schedule a set of jobs with specific deadlines and associated profits in such a way that the total profit is maximized while adhering to the deadlines.

Key Concepts:

1. Job Sequencing Problem: The Job Sequencing Problem is a classic optimization problem where we aim to schedule jobs in such a way that maximum profit is achieved. Each job has:

- An ID for identification.
- A profit associated with its completion.
- A deadline by which it must be completed.

The challenge is to assign jobs to time slots such that:

- Each job is completed within its deadline.
- The profit is maximized.
- Only one job can be assigned to a time slot.

2. **Greedy Approach:** The problem is solved using a greedy algorithm, which involves sorting the jobs by their profits in descending order and allocating each job to the latest available time slot before its deadline. If no such slot is available, the job is skipped.

3. **Sequential Data Structures:** To manage job data and time slots dynamically, linked lists can be used.

- Singly Linked List: Maintains a sequence of jobs, supporting efficient insertion, deletion, and traversal.
- Doubly Linked List: Provides bi-directional traversal, making it easier to update or reverse the schedule if needed.

4. **Applications:**

- Task scheduling in operating systems.
- Project management and resource allocation.
- Industrial automation for process optimization.

Job Scheduling Algorithm

Set of jobs with deadlines and profits are taken as an input with the job scheduling algorithm and scheduled subset of jobs with maximum profit are obtained as the final output.

Algorithm

Step1 − Find the maximum deadline value from the input set of jobs.

Step2 − Once, the deadline is decided, arrange the jobs in descending order of their profits.

Step3 − Selects the jobs with highest profits, their time periods not exceeding the maximum deadline.

Step4 − The selected set of jobs are the output.

Examples

Consider the following tasks with their deadlines and profits. Schedule the tasks in such a way that they produce maximum profit after being executed –

PCU

S. No.	Jobs	Deadlines	Profits
1	J1	2	20
2	J2	2	60
3	J3	1	40
4	J4	3	100
5	J5	4	80

Step 1: Find the maximum deadline value, dm, from the deadlines given.

dm = 4.

Step 2: Arrange the jobs in descending order of their profits.

S. No.	Jobs	Deadlines	Profits
1	J1	2	20
2	J2	2	60
3	J3	1	40
4	J4	3	100
5	J5	4	80

The maximum deadline, dm, is 4. Therefore, all the tasks must end before 4.

Choose the job with highest profit, J4. It takes up 3 parts of the maximum deadline.

Therefore, the next job must have the time period 1.

Total Profit = 100.

Step 3: The next job with highest profit is J5. But the time taken by J5 is 4, which exceeds the deadline by 3. Therefore, it cannot be added to the output set.

Step 4: The next job with highest profit is J2. The time taken by J5 is 2, which also exceeds the deadline by 1. Therefore, it cannot be added to the output set.

Step 5: The next job with higher profit is J3. The time taken by J3 is 1, which does not exceed the given deadline. Therefore, J3 is added to the output set.

Total Profit: 100 + 40 = 140

Step 6: Since, the maximum deadline is met, the algorithm comes to an end. The output set of jobs scheduled within the deadline are {J4, J3} with the maximum profit of 140.

PCU

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

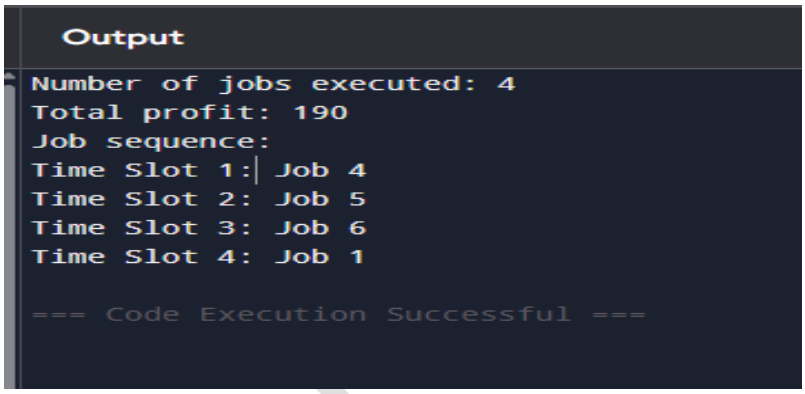
Name: **Roll No:**

- **Source Code :**

```
#include<iostream>
#include<algorithm> using namespace
std; struct Job {
  char id;
  int deadLine; int profit;
};
bool comp(Job j1, Job j2){
  return (j1.profit > j2.profit); //compare jobs based on profit
}
int min(int a, int b){ return (a<b)?a:b;
}
int main(){
  Job jobs[] = { { 'a', 2, 100 },
 { 'b', 2, 20 },
 { 'c', 1, 40 },
 { 'd', 3, 35 },
{ 'e', 1, 25 }
};
  int n = 5;
  cout << "Following is maximum profit sequence of Jobs: "<<"\n"; sort(jobs, jobs+n,
  comp); //sort jobs on profit
  int jobSeq[n]; // To store result (Sequence of jobs) bool slot[n];
  // To keep track of free time slots
  for (int i=0; i<n; i++)
```

```
    slot[i] = false; //initially all slots are free for (int i=0;
  i<n; i++) { //for all given jobs
    for (int j=min(n, jobs[i].deadLine)-1; j>=0; j--) { //search from last free slot if
      (slot[j]==false) {
        jobSeq[j] = i; // Add this job to job sequence slot[j] = true;
        // mark this slot as occupied break;
      }}}
  for (int i=0; i<n; i++) if (slot[i])
      cout << jobs[jobSeq[i]].id << " "; //display the sequence
}
```

- **Output:**



Conclusion:

Job sequencing using deadlines is a crucial problem-solving technique for maximizing profits in task scheduling. By utilizing efficient algorithms like greedy approaches or advanced methods like priority queues and disjoint sets, we can assign jobs to time slots dynamically while adhering to deadlines. This experiment enhances understanding of sequential data structures and equips students with practical knowledge in implementing scheduling algorithms using concepts like linked lists. It demonstrates how computational efficiency and data management play a vital role in solving real-world optimization problems.

Questions:-

1) What is the Job Sequencing problem, and what does it aim to solve?
2) Explain the concept of deadlines and profits in the Job Sequencing problem.
3) What is the time complexity of the greedy algorithm for the Job Sequencing problem?

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Assignment No. 4

Title: Write a program on Knapsack problem using Greedy method.

1) **Objective:**
   - Allocate the knapsack's limited capacity in a way that maximizes the total value.
   - Use the problem to illustrate the practical application and efficiency of greedy strategies in decision-making.
   - Show how partial inclusion (fractional items) can lead to better outcomes compared to rigid inclusion/exclusion approaches.
   - Address challenges in fields like logistics, finance, and resource allocation.
2) **Outcomes:**
   - Achieve the highest possible total value within the constraints of the knapsack's weight capacity.
   - Provide a solution with a time complexity of O(nlog fo n)O(n \log n), demonstrating computational efficiency.
   - Gain insights into solving optimization problems where divisibility is allowed.
   - Derive practical solutions for challenges in inventory management, investment strategies, and transportation logistics.
3) **Aim:**

   The aim of the Fractional Knapsack Problem is to maximize the total value of items that can be placed into a knapsack with a specific weight capacity by allowing fractional parts of items to be included. It seeks to find the most efficient way to use limited resources for maximum profit.
4) **Theory:**
   - A knapsack with a weight limit WW.
   - nn items, each having a weight wiw_i and a value viv_i.

   The objective is to maximize the total value of items in the knapsack while staying within its weight capacity. Unlike the 0/1 variant, you can divide items to include only a fraction, as long as it fits in the knapsack.

   Approach

   The problem is solved using a **greedy algorithm**, which operates as follows:

   1. Compute the **value-to-weight ratio** (vi/wiv_i / w_i) for each item.

PCU

2. **Sort items** in descending order of this ratio.
3. Add items to the knapsack, starting with the highest ratio:
   - If an entire item fits, include it fully.
   - If not, include the fraction that fits.

The Problem Statement

The knapsack problem involves:

- A knapsack with a maximum weight capacity WW.
- nn items, each having:
  - A weight wiw_i.
  - A value viv_i.

The goal is to maximize the total value of items placed in the knapsack without exceeding its weight capacity. Unlike the 0/1 Knapsack Problem, where you can either include or exclude an item entirely, in the *Fractional Knapsack Problem*, you can break an item into smaller fractions, thus allowing a portion of it to be included in the knapsack.

The Algorithm:

The greedy approach is optimal for the fractional knapsack problem. Here's how it works step by step:

Step 1: Calculate the value-to-weight ratio

For each item ii, compute:

Ratioi=viwi\text{Ratio}_i = \frac{v_i}{w_i}

This ratio determines the "profitability" of an item per unit of weight.

Step 2: Sort items

Sort all items in descending order of their value-to-weight ratio.

Step 3: Pick items

Iterate through the sorted list:

- If the item's weight wiw_i is less than or equal to the remaining capacity of the knapsack WW, take the entire item.
- If wi>Ww_i > W, take the fraction of the item that fits into the remaining capacity, i.e., W/wiW / w_i.
- Subtract the used weight from WW.

Step 4: Stop when capacity is full

Time Complexity

- Sorting the items based on value-to-weight ratio takes O(nlog n)O(n \log n).
- Iterating through the items to fill the knapsack takes O(n)O(n).
- Thus, the overall time complexity is O(nlog n)O (n \log n).

Example

Imagine you have the following items:

Item	Value(vv)	Weight (ww)	Value-to-Weight (v/wv/w)	Ratio
A	60	10	6	
B	100	20	5	
C	120	30	4	

Knapsack Capacity = 5050.

Solution:

1. Sort by value-to-weight ratio: A, B, C.
2. Add entire item A (w=10w = 10, v=60v = 60).
3. Add entire item B (w=20w = 20, v=100v = 100).
4. Add 2/3 of item C (w=20w = 20, v=80v = 80).

Total Value = 60 + 100 + 80 = 240.

Applications

- Financial portfolio management (optimizing investments).
- Supply chain logistics.
- Bandwidth allocation in networks.

Does this explanation shed enough light on the topic? I can dive deeper or provide pseudocode for implementation if you'd like!

Source Code:

```
#include<bits/stdc++.h>
using namespace std;
// Struct to represent an item with profit and weight
struct Item {
```

```
    int profit;
    int weight;
};
class Solution {
public:
    // Comparator function to sort items by profit/weight ratio
    bool static comp(Item a, Item b) {
        double r1 = (double) a.profit / (double) a.weight;
        double r2 = (double) b.profit / (double) b.weight;
        return r1 > r2;  // Return true if the ratio of item a is greater than item b
    }
    // Function to calculate the maximum profit we can get with fractional knapsack
    double fractionalKnapsack(int W, Item arr[], int n)
{
        // Sort items based on the profit/weight ratio
        sort(arr, arr + n, comp);
        int curWeight = 0;  // Current weight of knapsack
        double finalprofit = 0.0;  // Final profit we can achieve
        // Iterate through the sorted items
        for (int i = 0; i < n; i++)
{
            // If the current item can be fully added to the knapsack
            if (curWeight + arr[i].weight <= W) {
                curWeight = curWeight + arr[i].weight;
                finalprofit = finalprofit + arr[i].profit;  // Add the full profit of the item
            } else {
                // If the current item can't be fully added, take the fractional part
                int remain = W - curWeight;
                finalprofit = finalprofit + (arr[i].profit / (double) arr[i].weight) * (double) remain;
                break;  // Break as we have filled the knapsack
```

```
            }
        }
        return finalprofit;  // Return the maximum profit that can be carried
    }
};
int main() {
    int n = 3, weight = 50;  // Number of items and capacity of knapsack
    // Item profits and weights
    Item arr[n] = { {100,20},{60,10},{120,30} };
    Solution obj;
    // Calculate the maximum profit we can get with the fractional knapsack
    double ans = obj.fractionalKnapsack(weight, arr, n);
        // Output the result
    cout << "The maximum profit is " << setprecision(2) << fixed << ans;
        return 0;
}
```

Output:
The maximum profit is 240.00

PCU

Questions:-

1) What is the fractional knapsack problem, and how does it differ from the 0/1 knapsack problem?
2) Explain the concept of "greedy approach" used in solving the fractional knapsack problem.
3) What is the time complexity of the greedy algorithm for the fractional knapsack problem?

PCET's Pimpri Chinchwad University Learn | Grow | Achieve	**PIMPRI CHINCHWAD UNIVERSITY** **School of Engineering and Technology**

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Assignment No. 5

Title: Write a program to implement Bellman-Ford Algorithm using Dynamic Programming and verify the time complexity

5) **Objective:**

- Implement Bellman-Ford using dynamic programming.
- Verify correctness of shortest paths even with negative weights.
- Detect negative weight cycles.
- Analyze and confirm time complexity.

6) **Outcomes:**

- Successfully computes shortest paths from a single source.
- Works correctly even with negative edge weights.
- Identifies negative weight cycles.
- Confirms theoretical time complexity O(V·E).

7) **Aim:**

To implement the **Bellman-Ford Algorithm** using dynamic programming principles for finding the shortest paths from a single source to all vertices in a weighted graph (including graphs with negative edge weights), and to verify its **time complexity**.

8) **Theory:**

- Bellman-**Ford Algorithm** is a **single-source shortest path algorithm**.
- Unlike Dijkstra's algorithm, it can handle **negative edge weights**.
- It works by **relaxing all edges repeatedly**:

- For a graph with V vertices, we perform V−1 iterations.
- Each iteration updates the shortest distance estimates.

Approach

- **Dynamic Programming View**:
  - Define dp[k][v] = shortest distance to vertex v using at most k edges.
  - Recurrence relation:

$$dp[k][v]=\min(dp[k-1][v],\min_{(u,v)\in E}(dp[k-1][u]+w(u,v)))$$

- Final answer after V−1 relaxations gives shortest paths.

The Problem Statement

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Design and implement a program to compute the **shortest paths from a single source vertex to all other vertices in a weighted graph** using the **Bellman-Ford Algorithm**. The algorithm should:

- Handle **negative edge weights**.
- Detect **negative weight cycles**.
- Be implemented using **dynamic programming principles**.
- Verify the **time complexity** of the algorithm.

The Algorithm:

Step 1: Initialization

- Let `dist[v]` represent the shortest distance from the source to vertex `v`.
- Set `dist[source] = 0` and all other `dist[v] = ∞`.

1. **Step 2: Relaxation (Dynamic Programming Recurrence)**
   - For each iteration i=1 to V−1:
     - For every edge (u,v) with weight w:

     dist[v]=min(dist[v],dist[u]+w)

Step 3: Negative Cycle Detection

- Perform one more relaxation step.
- If any distance can still be updated, then a **negative weight cycle** exists.

Step 4: Output

- Print shortest distances from the source to all vertices.
- Report if a negative cycle is detected.

Time Complexity

- **Relaxation step**: O(E)
- Repeated for V−1 iterations: O(V·E)
- **Final complexity**:

  O(V·E)
- **Space complexity**: O(V)

Example Graph (Directed,Weighted)

We'll use **4 vertices**: {0, 1, 2, 3} And the following edges:

- 0→1 (weight = 1)
- 0→2 (weight = 4)

- 1→2 (weight = -3)
- 1→3 (weight = 2)
- 2→3 (weight = 3)

Imagine you have the following Graph

```
      (0)
      / \
    1/   \4
    /     \
(1)---->(2)
  | 2     |
  v       v
(3)<-----3
```

Solution:

- From **0 to 1**: weight 1
- From **0 to 2**: weight 4
- From **1 to 2**: weight -3 (negative edge)
- From **1 to 3**: weight 2
- From **2 to 3**: weight 3
  - It contains a **negative edge** (1 → 2 with weight -3).
  - It does **not** contain a negative cycle, so Bellman-Ford will compute valid shortest paths.
  - If we added another edge like (3 → 1, -5), that would create a **negative cycle**.

Expected Shortest Path Results (from source = 0)

- Distance to 0 = 0
- Distance to 1 = 1
- Distance to 2 = -2 (via 0 → 1 → 2)
- Distance to 3 = 1 (via 0 → 1 → 3)

Applications

- Financial portfolio management (optimizing investments).
- Supply chain logistics.
- Bandwidth allocation in networks.

PCU

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Source Code:

```
class Edge:
 def __init__(self, u, v, w):
    self.u = u
    self.v = v
    self.w = w
def bellman_ford(vertices, edges, source):
 # Step 1: Initialize distances
 dist = [float("inf")] * vertices
 dist[source] = 0
 # Step 2: Relax edges repeatedly
 for _ in range(vertices - 1):
    for edge in edges:
       if dist[edge.u] != float("inf") and dist[edge.u] + edge.w < dist[edge.v]:
          dist[edge.v] = dist[edge.u] + edge.w
 # Step 3: Check for negative weight cycles
 for edge in edges:
    if dist[edge.u] != float("inf") and dist[edge.u] + edge.w < dist[edge.v]:
       print("Graph contains a negative weight cycle!")
       return None
 return dist
# Example usage
Edges  = [
 Edge (0, 1, 1),
 Edge (0, 2, 4),
 Edge (1, 2, -3),
 Edge (1, 3, 2),
 Edge (2, 3, 3)
  ]
vertices  = 4
```

```
source  = 0
distances = bellman_ford(vertices, edges, source)
if distances:
    print ("Shortest distances from source:", distances)
```

Output:

Shortest distances from source: [0, 1, -2, 1]

Questions:-

1] Write programs for both **Bellman-Ford and Dijkstra algorithms** and compare their results for the same graph.

2] Extend the Bellman-Ford program to **print the actual shortest path** from the source vertex to each vertex.

3] Modify the **Bellman-Ford Algorithm** program to **detect whether the graph contains a negative weight cycle**.

PCU

PCET's Pimpri Chinchwad University Learn | Grow | Achieve	**PIMPRI CHINCHWAD UNIVERSITY** **School of Engineering and Technology**

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Assignment No. 6

Title: Write a program to solve the travelling salesman problem and to print the path and the cost using Dynamic Programming

9) Objective:

1. Understand the concept of **Travelling Salesman Problem**.
2. Apply **Dynamic Programming technique** to optimization problems.
3. Implement **Held–Karp algorithm** using Python.
4. Compute **minimum travelling cost** among cities.
5. Print the **optimal path followed by the salesman**.

Outcomes:

 Understand **the concept of the Travelling Salesman Problem (TSP)** and its importance in Optimization and graph problems.

 Apply **the Dynamic Programming technique** to solve complex optimization problems efficiently

 Compute **the minimum travelling cost** required to visit all cities exactly once and return to the starting city.

 Determine and print the optimal path followed by the salesman that results in the minimum cost.

10) Aim:

- To design and implement a program to solve the **Travelling Salesman Problem (TSP)** using **Dynamic Programming** and print the **minimum cost and optimal path**.

1) Theory:

The **Travelling Salesman Problem (TSP)** is a classical optimization problem.

Given:

- A set of **cities**
- Distance between every pair of cities

Goal:

- Visit each city **exactly once**
- Return to the **starting city**
- Minimize the **total travelling cost**

If there are **n cities**, the brute force solution checks **all permutations**, giving:

O(n!)O(n!)O(n!)

PCU

Dynamic Programming improves this using the **Held-Karp algorithm** with complexity:

O(n2×2n)O(n^2 \times 2^n)O(n2×2n)

The algorithm uses:

- **Bitmasking** → to track visited cities
- **Memoization** → to store intermediate results

This reduces redundant calculations.

 The **Travelling Salesman Problem (TSP)** is a well-known optimization problem in computer science. Given:

- A set of **cities**
- The **distance between every pair of cities**

The objective is to:

- Start from a city
- Visit every other city **exactly once**
- Return to the **starting city**
- Minimize the **total travel cost**

Approach

 Represent cities using an **adjacency matrix**.

 Use **bit masking** to track visited cities.

 Start from city **0**.

 Recursively compute the cost of visiting remaining cities.

 Store intermediate results in a **DP table**.

 finally return to the starting city.

- **Dynamic Programming View**:

The Problem Statement

Write a program to determine the **shortest possible route** that visits each city exactly once and returns to the starting city using **Dynamic Programming**.

The Algorithm:

 Start

 Define distance matrix dist

 Define recursive function tsp(mask, pos)

PCU

- If all cities are visited:
  - Return cost to return to starting city
- for each city not visited:
  - Calculate cost of visiting that city
- Store minimum cost
- Return minimum cost
- Print optimal path and cost
- Stop

Procedure

1. Define the number of cities.
2. Create a **distance matrix** representing travel cost between cities.
3. Use **Dynamic Programming with bitmasking** to store visited cities.
4. Start travelling from **city 0**.
5. Recursively visit remaining cities.
6. Calculate cost for each possible route.
7. Select the **minimum cost route**.
8. Display the **optimal path**.
9. Display the **minimum travelling cost**

Output

Minimum Travelling Cost = 80

Optimal Path:

0 → 1 → 3 → 2 → 0

Time Complexity

Method	Time Complexity
Brute Force	O(n!)
Dynamic Programming	$O(n^2 \times 2^n)$

Dynamic Programming is much more efficient than brute force for larger inputs.

- **Space complexity**:

  The space complexity of the **Dynamic Programming TSP algorithm** is:

  O(n×2n)O(n \times 2^n)O(n×2n)

PCU

Where:

- **n** = number of cities
- 2^n = possible subsets of cities

Space is used for:

- **DP table**
- **Recursion stack**

Example

City	A	B	C	D
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

Solution:

Step-by-Step Possible Paths

Starting from **City A**.

1. **A → B → C → D → A**
   Cost = 10 + 35 + 30 + 20 = **95**
2. **A → B → D → C → A**
   Cost = 10 + 25 + 30 + 15 = **80**
3. **A → C → B → D → A**
   Cost = 15 + 35 + 25 + 20 = **95**
4. **A → C → D → B → A**
   Cost = 15 + 30 + 25 + 10 = **80**
5. **A → D → B → C → A**
   Cost = 20 + 25 + 35 + 15 = **95**
6. **A → D → C → B → A**
   Cost = 20 + 30 + 35 + 10 = **95**

Optimal Path:

A → B → D → C → A

Cost Calculation:

10 + 25 + 30 + 15 = **80**

PCU

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

City Number

A	0
B	1
C	2
D	3

City	A	B	C	D
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

Visited Set	Ending City	Cost
{A,B}	B	10
{A,C}	C	15
{A,D}	D	20

Visited Set	Ending City	Cost Calculation	Cost
{A,B,C}	C	A→B→C = 10 + 35	45
{A,B,C}	B	A→C→B = 15 + 35	50
{A,B,D}	D	A→B→D = 10 + 25	35
{A,B,D}	B	A→D→B = 20 + 25	45
{A,C,D}	D	A→C→D = 15 + 30	45
{A,C,D}	C	A→D→C = 20 + 30	50

Visited Set	Ending City	Cost Calculation	Cost
{A,B,C,D}	D	A→B→C→D = 10+35+30	75
{A,B,C,D}	C	A→B→D→C = 10+25+30	65
{A,B,C,D}	B	A→C→D→B = 15+30+25	70

Path	Cost
A → B → C → D → A	10 + 35 + 30 + 20 = 95
A → B → D → C → A	10 + 25 + 30 + 15 = **80**
A → C → B → D → A	15 + 35 + 25 + 20 = 95
A → C → D → B → A	15 + 30 + 25 + 10 = **80**

PCU

Path	Cost
A → D → B → C → A	20 + 25 + 35 + 15 = 95
A → D → C → B → A	20 + 30 + 35 + 10 = 95

Applications

- **Logistics and Delivery Routing**

It is used by courier and delivery companies to determine the **shortest route for delivering packages** to multiple locations and returning to the starting point.

- **Transportation Planning**

Helps in planning the **most efficient travel routes** for buses, taxis, and transportation networks to reduce travel time and fuel cost.

- **Tour Planning**

Used by travel agencies to design **optimal tour routes** for tourists visiting multiple cities while minimizing travel distance.

Source Code:

```
import itertools
def tsp_dynamic(graph):
    n = len(graph)
    cities = range(1, n)
    min_cost = float('inf')
    best_path = []
    for perm in itertools.permutations(cities):
        current_cost = 0
        k = 0
        path = [0]
        for j in perm:
            current_cost += graph[k][j]
            k = j
            path.append(j)
        current_cost += graph[k][0]
        path.append(0)
```

PCU

```
            if current cost < min_cost:
                min_cost = current cost
                best_path = path
        return best_path, min_cost
graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]
path, cost = tsp_dynamic(graph)
print("Optimal Path:", path)
print("Minimum Cost:", cost)
Output:
Optimal Path: [0, 1, 3, 2, 0]
Minimum Cost: 80
```

City0 → City1 → City3 → City2 → City0

Total cost = **80**

Questions:-

1] What technique is used in Dynamic Programming for TSP?

2] Where is TSP used in real-world applications?

3] What is the base condition in the TSP DP algorithm?

PCU

PCET's Pimpri Chinchwad University Learn | Grow | Achieve	**PIMPRI CHINCHWAD UNIVERSITY** **School of Engineering and Technology**

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Assignment No. 7

Title: Write a recursive program to find the solution of placing n queens on the chessboard so that no two queens attack each other using Backtracking

Objective:

• Understand the concept of recursion and backtracking.

• Apply backtracking to solve the N-Queens Problem.

• Learn how invalid choices are rejected to reduce the search space.

• Demonstrate the algorithm with an example.

Outcomes:

• A working program that solves the N-Queens Problem using Backtracking.

• Reduced unnecessary checking by rejecting unsafe queen placements.

• Clear understanding of recursion and backtracking in constraint satisfaction problems.

Aim: To implement the N-Queens Problem using the Backtracking technique and analyze its efficiency in finding valid queen arrangements on the chessboard.

2) Theory:

The **N-Queens Problem** is a classic problem in combinatorial optimization and constraint satisfaction.

• The task is to place **n queens** on an **n × n chessboard**.

• No two queens should attack each other.

• A queen can attack another queen in the same **row, column, or diagonal**.

Approach

• Place one queen in each row.

• For each row, try placing the queen in every column.

• Check whether the position is safe.

• If safe, place the queen and move to the next row recursively.

• If no column is safe, backtrack and remove the previously placed queen.

• Continue until all queens are placed.

The Problem Statement

Given an integer **n**, place **n queens** on an **n × n chessboard** such that no two queens attack each other.

The Algorithm:

Procedure:

PCU

- Start from the first row.
- Try placing a queen in each column of that row.
- Check whether the position is safe:
- No queen in the same column
- No queen in the left diagonal
- No queen in the right diagonal
- If the position is safe, place the queen.
- Recursively repeat the process for the next row.
- If placing a queen leads to no solution, remove it and try the next column.
- Repeat until all queens are placed successfully.

Time Complexity

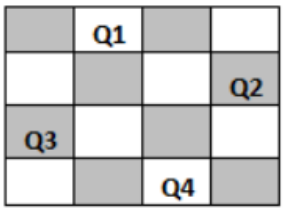
- Worst case: **O(N!)**, as many possible arrangements are explored.
- Backtracking reduces the search by rejecting invalid placements early.

Space Complexity:

- **O(N × N)** for storing the chessboard.
- Additional recursion stack space: **O(N)**.

Example

For **N = 4**, one possible solution is:



Solution 1

Solution:

For **N = 4**

One valid arrangement is:

- Row 1 → Column 2
- Row 2 → Column 4
- Row 3 → Column 1
- Row 4 → Column 3

PCU

This arrangement ensures that no two queens attack each other.

Applications

• Solving constraint satisfaction problems.

• Game solving and puzzle design.

• Scheduling and resource allocation.

• Artificial Intelligence and search algorithms.

• Optimization problems using backtracking.

Source Code:

```
def print_board(board, n):
    for i in range(n):
        for j in range(n):
            print(board[i][j], end=" ")
        print()
def is_safe(board, row, col, n):
    # Check same column
    for i in range(row):
        if board[i][col] == 1:
            return False
    # Check left diagonal
    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1
    # Check right diagonal
    i, j = row - 1, col + 1
    while i >= 0 and j < n:
        if board[i][j] == 1:
            return False
```

PCU

```
        i -= 1
        j += 1
    return True
def solve_n_queens(board, row, n):
    if row == n:
        return True
    for col in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = 1
            if solve_n_queens(board, row + 1, n):
                return True
            board[row][col] = 0   # Backtrack
    return False
# Example
n = 4
board = [[0 for _ in range(n)] for _ in range(n)]
if solve_n_queens(board, 0, n):
    print("Solution exists:")
    print_board(board, n)
else:
    print("No solution exists")
```

Output:

For **N = 4**

Solution exists:

0 1 0 0

0 0 0 1

1 0 0 0

0 0 1 0

PCU

PIMPRI CHINCHWAD UNIVERSITY
School of Engineering and Technology

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Questions:-

1] Solve the N-Queens problem using Backtracking for **N = 4**.

2] Explain how backtracking helps reduce the search space in the N-Queens Problem.

3] Compare Backtracking with Brute Force for solving the N-Queens Problem.

PCU

PCET's Pimpri Chinchwad University Learn | Grow | Achieve	**PIMPRI CHINCHWAD UNIVERSITY** **School of Engineering and Technology**

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

Assignment No. 8

Title: Write a program for Knapsack problem using Brach and Bound

Objective:

- Understand the concept of **Branch and Bound** in combinatorial optimization.
- Apply it to solve the **Knapsack Problem** efficiently.
- Learn how bounding functions help prune the search space.
- Demonstrate the algorithm with an example.

Outcomes:

- A working program that solves the Knapsack problem using Branch and Bound.
- Reduced computational effort compared to brute force.
- Clear understanding of bounding and pruning in optimization problems.

Aim: To implement the **0/1 Knapsack Problem** using the **Branch and Bound technique** and analyze its efficiency compared to other approaches.

2) Theory:

The **0/1 Knapsack Problem** is a classic combinatorial optimization problem.

- You have **n items**, each with a **profit** and a **weight**.
- You must select items to maximize profit without exceeding the knapsack's capacity.
- Unlike fractional knapsack, you cannot take fractions of items — only include or exclude each item.

Approach

- Sort **items** by profit-to-weight ratio.
- Initialize **root node** with level = -1, profit = 0, weight = 0.
- Use a **priority queue** (max-heap) to explore nodes with highest bound first.
- At each node:
  - Generate child nodes (include/exclude next item).
  - Compute bound for each child.
  - If bound > current best profit, add to queue.
- Continue until queue is empty.
- The best profit found is the optimal solution.

PCU

Academic Year:2025-2026 **Sem:** 6

Course: Design and analysis of algorithms Laboratory **Course code**: UBTCE313

Name: **Roll No:**

The Problem Statement

Given **n items**, each with a **weight** and a **profit**, and a knapsack with **capacity W**, select items to maximize the total profit without exceeding the capacity. Formally:

Maximize ∑pixi subject to ∑wixi≤W, where xi∈{0,1}.

The Algorithm:

Procedure:

- **Sort items** by profit/weight ratio.
- **Use a priority queue** to explore nodes (states).
- Each node represents a decision: include or exclude an item.
- Compute **upper bound** (maximum possible profit from this node).
- If the bound is less than the best solution so far → **prune**.
- Continue until all promising nodes are explored.

Time Complexity

- Worst case: O(2n) (explores all subsets).
- Average case: Much better due to pruning; depends on bounding efficiency.

Space Complexity:

- O(n) for storing items + O(2n) in worst case for priority queue.
- Practically, space is reduced due to pruning.

Example

- Items:

Items	Profit	Weight
1	60	10
2	100	20
3	120	30

- Capacity W=50.
- Optimal solution: Items 2 and 3 → Profit = 220

Solution:

- Items:
  - Item1: Profit=60, Weight=10

PCU

- Item2: Profit=100, Weight=20
- Item3: Profit=120, Weight=30

Optimal selection: Items with profits 60 and 40 (weights 5+4=9). **Maximum Profit = 100.**

Applications

- Resource allocation in project management.
- Cargo loading optimization.
- Budget planning in finance.
- Memory allocation in operating systems.
- Portfolio selection in investment.

Source Code:

```
import heapq
class Node:
    def __init__(self, level, profit, weight, bound, items):
        self.level = level
        self.profit = profit
        self.weight = weight
        self.bound = bound
        self.items = items
    def __lt__(self, other):
        return self.bound > other.bound  # max-heap behavior
def bound(node, n, W, weights, profits):
    if node.weight >= W:
        return 0
    profit_bound = node.profit
    j = node.level + 1
    totweight = node.weight
    while j < n and totweight + weights[j] <= W:
        totweight += weights[j]
        profit_bound += profits[j]
        j += 1
```

PCU

```
        if j < n:
            profit_bound += (W - totweight) * profits[j] / weights[j]
        return profit_bound
    def knapsack_branch_bound(W, weights, profits, n):
        items = sorted(range(n), key=lambda i: profits[i]/weights[i], reverse=True)
        weights = [weights[i] for i in items]
        profits = [profits[i] for i in items]
        Q = []
        u = Node(-1, 0, 0, 0, [])
        u.bound = bound(u, n, W, weights, profits)
        heapq.heappush(Q, u)
        maxProfit = 0
        bestItems = []
        while Q:
            u = heapq.heappop(Q)
            if u.bound > maxProfit and u.level < n-1:
                v = Node(u.level+1, u.profit+profits[u.level+1],
                        u.weight+weights[u.level+1], 0, u.items+[items[u.level+1]])
                if v.weight <= W and v.profit > maxProfit:
                    maxProfit = v.profit
                    bestItems = v.items
                v.bound = bound(v, n, W, weights, profits)
                if v.bound > maxProfit:
                    heapq.heappush(Q, v)
                v = Node(u.level+1, u.profit, u.weight, 0, u.items)
                v.bound = bound(v, n, W, weights, profits)
                if v.bound > maxProfit:
                    heapq.heappush(Q, v)
        return maxProfit, bestItems
    # Example
```

```
profits = [60, 100, 120]
weights = [10, 20, 30]
W = 50
n = len(profits)
maxProfit, bestItems = knapsack_branch_bound(W, weights, profits, n)
print("Maximum Profit:", maxProfit)
print("Items selected:", bestItems)
import heapq
class Node:
    def __init__(self, level, profit, weight, bound, items):
        self.level = level
        self.profit = profit
        self.weight = weight
        self.bound = bound
        self.items = items
    def __lt__(self, other):
        return self.bound > other.bound  # max-heap behavior
def bound(node, n, W, weights, profits):
    if node.weight >= W:
        return 0
    profit_bound = node.profit
    j = node.level + 1
    totweight = node.weight
    while j < n and totweight + weights[j] <= W:
        totweight += weights[j]
        profit_bound += profits[j]
        j += 1
    if j < n:
        profit_bound += (W - totweight) * profits[j] / weights[j]
    return profit_bound
```

```
def knapsack_branch_bound(W, weights, profits, n):
    items = sorted(range(n), key=lambda i: profits[i]/weights[i], reverse=True)
    weights = [weights[i] for i in items]
    profits = [profits[i] for i in items]
    Q = []
    u = Node(-1, 0, 0, 0, [])
    u.bound = bound(u, n, W, weights, profits)
    heapq.heappush(Q, u)
    maxProfit = 0
    bestItems = []
    while Q:
        u = heapq.heappop(Q)
        if u.bound > maxProfit and u.level < n-1:
            v = Node(u.level+1, u.profit+profits[u.level+1],
                    u.weight+weights[u.level+1], 0, u.items+[items[u.level+1]])
            if v.weight <= W and v.profit > maxProfit:
                maxProfit = v.profit
                bestItems = v.items
            v.bound = bound(v, n, W, weights, profits)
            if v.bound > maxProfit:
                heapq.heappush(Q, v)
            v = Node(u.level+1, u.profit, u.weight, 0, u.items)
            v.bound = bound(v, n, W, weights, profits)
            if v.bound > maxProfit:
                heapq.heappush(Q, v)
    return maxProfit, bestItems
# Example
profits = [60, 100, 120]
weights = [10, 20, 30]
W = 50
```

```
n = len(profits)
maxProfit, bestItems = knapsack_branch_bound(W, weights, profits, n)
print("Maximum Profit:", maxProfit)
print("Items selected:", bestItems)
```

Output:

- Profits = [40, 35, 60, 55]
- Weights = [4, 3, 5, 6]
- Capacity = 10

Optimal selection: Items with profits 60 and 40 (weights 5+4=9). **Maximum Profit = 100.**

Questions:-

1] Solve the knapsack problem using Branch and Bound for:
- Profits = [40, 35, 60, 55]
- Weights = [4, 3, 5, 6]
- Capacity = 10

2] Explain how bounding helps reduce the search space.
3] Compare Branch and Bound with Dynamic Programming for Knapsack.

PCU